

ACM 算法竞赛模板完全手册

目录

第一部分：基础模板与STL

1. [常用头文件与宏定义](#)
2. [STL 常用容器与函数](#)
3. [STL 进阶用法](#)

第二部分：核心数据结构

4. [数据结构](#)

第三部分：图论算法

5. [图论](#)

第四部分：搜索与DP

6. [搜索算法](#)
7. [动态规划](#)

第五部分：数学与字符串

8. [数学与数论](#)
9. [字符串](#)

第六部分：技巧与实战

10. [常用函数与技巧](#)
11. [洛谷模板题验证列表](#)
12. [比赛常用技巧](#)

第一部分：基础模板与STL

常用头文件与宏定义

标准模板

```
#include <bits/stdc++.h>
using namespace std;

#define IOS ios::sync_with_stdio(false); cin.tie(nullptr); cout.tie(nullptr)
```

```
#define endl '\n'
#define int long long
#define ld long double
#define pb push_back
#define PII pair<int,int>
#define mp make_pair
#define ull unsigned long long
#define i128 __int128
#define all(x) (x).begin(), (x).end()
#define fi first
#define se second

const int INF = 1e9 + 10;
const int LINF = 1e18 + 10;
const ld PI = acos(-1.0);
const ld EPS = 1e-9;

void Asanagi()
{
    // 主逻辑
}

signed main()
{
    IOS;
    int t = 1;
    // cin >> t;
    while (t--)
    {
        Asanagi();
    }
    return 0;
}
```

常用类型别名

```
using ll = long long;
using ld = long double;
using pii = pair<int, int>;
using pll = pair<ll, ll>;
using vi = vector<int>;
using vl = vector<ll>;
using vb = vector<bool>;
using vs = vector<string>;
```

STL 常用容器与函数

1. 序列容器

vector

```

vector<int> v;           // 定义
vector<int> v(n);        // 大小为n, 值默认0
vector<int> v(n, 5);     // 大小为n, 值全为5
v.push_back(10);         // 尾部插入
v.pop_back();            // 删除尾部
v.size();                // 大小
v.empty();               // 是否为空
v.clear();               // 清空
v.resize(n);             // 改变大小
v.front(); v.back();     // 首尾元素
sort(v.begin(), v.end()); // 排序
reverse(v.begin(), v.end()); // 反转

```

deque (双端队列)

```

deque<int> dq;
dq.push_front(1);        // 头部插入
dq.push_back(2);         // 尾部插入
dq.pop_front();          // 头部删除
dq.pop_back();           // 尾部删除
dq.front(); dq.back();   // 首尾元素

```

2. 关联容器

set / multiset

```

set<int> s;
s.insert(10);            // 插入
s.erase(10);             // 删除
s.find(10);              // 查找, 返回迭代器
s.count(10);             // 计数 (set中为0或1)
s.lower_bound(10);        // 第一个 >= 10 的迭代器
s.upper_bound(10);        // 第一个 > 10 的迭代器

```

map / multimap

```
map<string, int> mp;
mp["key"] = 5;           // 插入/修改
mp.find("key");          // 查找
mp.count("key");         // 是否存在
mp.erase("key");         // 删除
for (auto &[k, v] : mp) { ... } // 遍历 (C++17)
```

3. 栈和队列

```
stack<int> stk;
stk.push(10); stk.pop();
int top = stk.top();

queue<int> qe;
qe.push(10); qe.pop();
int front = qe.front();

priority_queue<int> pq;           // 大根堆
priority_queue<int, vector<int>, greater<int>> pq; // 小根堆
pq.push(10); pq.pop();
int top = pq.top();
```

STL 进阶用法

1. `__gnu_pbds` (扩展平衡树 / 有序哈希表)

GNU扩展库提供了比STL更强大的数据结构。

tree (有序平衡树)

`tree` 可以实现类似 `set` 的功能，但支持**顺序统计**（查找第k大元素）。

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace std;
using namespace __gnu_pbds;

// 定义tree: 支持顺序统计的set
template<typename T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update>;

int main() {
    ordered_set<int> os;
```

```

os.insert(10);
os.insert(20);
os.insert(30);

// 查找第k小的元素 (0-indexed)
cout << *os.find_by_order(0) << endl; // 输出: 10 (第1小)
cout << *os.find_by_order(2) << endl; // 输出: 30 (第3小)

// 查找某个元素的排名 (0-indexed, 即比它小的元素个数)
cout << os.order_of_key(25) << endl; // 输出: 2 (25的排名是第2)

return 0;
}

```

gp_hash_table (高性能哈希表)

比 `unordered_map` 更快的哈希表实现。

```

#include <ext/pb_ds/hash_policy.hpp>
using namespace __gnu_pbds;

gp_hash_table<int, int> ght;
ght[1] = 100;
ght[2] = 200;

cout << ght[1] << endl; // 输出: 100

```

2. `std::bitset` 的常见用法

`bitset` 是固定大小的位集，比 `bool` 数组更节省空间，且支持位运算。

```

bitset<100> bs1; // 全部初始化为0
bitset<100> bs2(5); // 二进制表示: 101
bitset<100> bs3("1010"); // 从字符串初始化

bs1.set(5); // 设置第5位为1
bs1.reset(5); // 重置第5位为0
bs1.flip(5); // 翻转第5位

cout << bs1.count() << endl; // 返回1的个数
cout << bs1.any() << endl; // 是否有1
cout << bs1.none() << endl; // 是否全0

// 位运算
bitset<100> bs4 = bs1 & bs2; // 与
bitset<100> bs5 = bs1 | bs2; // 或
bitset<100> bs6 = bs1 ^ bs2; // 异或

```

3. `std::complex` 的用法 (计算几何)

`complex` 可以方便地进行复数运算，在计算几何中常用于表示二维点。

```
using Point = complex<double>;
using Vector = complex<double>;

Point p1(1.0, 2.0); // 点 (1, 2)
Point p2(3.0, 4.0); // 点 (3, 4)

Vector v = p2 - p1; // 向量 p1->p2
double len = abs(v); // 向量的模长
double dist = abs(p1 - p2); // 两点距离
double angle = arg(v); // 向量的幅角 (弧度)

// 点积和叉积
double dot = real(conj(v1) * v2);
double cross = imag(conj(v1) * v2);

// 旋转
Vector v_rot = v * polar(1.0, PI/4); // 逆时针旋转45度
```

4. 常用的STL算法

`nth_element`

快速找到第n小的元素（部分排序）。

```
vector<int> arr = {9, 5, 2, 8, 1, 7, 3, 6, 4};
nth_element(arr.begin(), arr.begin() + 4, arr.end());
cout << "第5小的元素是: " << arr[4] << endl; // 输出: 5
```

`partial_sum`

计算部分和。

```
vector<int> arr = {1, 2, 3, 4, 5};
vector<int> prefix(arr.size());
partial_sum(arr.begin(), arr.end(), prefix.begin());
// prefix = {1, 3, 6, 10, 15}
```

`iota`

填充递增序列。

```
vector<int> arr(10);
iota(arr.begin(), arr.end(), 0); // 填充 0, 1, 2, ..., 9
```

next_permutation

生成下一个排列。

```
vector<int> arr = {1, 2, 3};
do {
    for (int x : arr) cout << x << " ";
    cout << endl;
} while (next_permutation(arr.begin(), arr.end()));
```

5. string 的进阶用法

```
string s = "Hello, World!";

cout << s.substr(0, 5) << endl; // 输出: Hello
cout << s.substr(7) << endl;    // 输出: World!

// 查找
cout << s.find("World") << endl; // 输出: 7
cout << s.find('x') << endl;     // 输出: string::npos (没找到)

// 字符串转数字
string num = "123";
int x = stoi(num); // 123
long y = stol(num); // 123
long long z = stoll(num); // 123

// 数字转字符串
string s4 = to_string(12345); // "12345"
```

6. stringstream 的用法 (快速解析输入)

```
string line = "42 3.14 hello 100";
stringstream ss(line);

int a;
double b;
string c;
int d;

ss >> a >> b >> c >> d;
```

```
cout << a << " " << b << " " << c << " " << d << endl;
// 输出: 42 3.14 hello 100
```

7. tuple 和 tie / structured bindings

```
// tuple
tuple<int, string, double> t(42, "hello", 3.14);
cout << get<0>(t) << endl; // 42
cout << get<1>(t) << endl; // "hello"

// tie
int a; string b; double c;
tie(a, b, c) = t;

// structured bindings (C++17)
auto [x, y, z] = t;
```

8. 竞赛中常用的 lambda 表达式模式

```
// 基本lambda
auto add = [](int a, int b) { return a + b; };

// 捕获外部变量
int base = 10;
auto add_base = [base](int x) { return x + base; };

// 在STL算法中使用
vector<int> arr = {3, 1, 4, 1, 5, 9, 2, 6};
sort(arr.begin(), arr.end(), [](int a, int b) {
    return a > b; // 降序
});

// 递归lambda
function<int(int)> fib = [&fib](int n) -> int {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
};
```

第二部分：核心数据结构

数据结构

1. 并查集 (DSU)

基础版

```

struct DSU {
    vector<int> fa, siz;

    DSU(int n) : fa(n + 1), siz(n + 1, 1) {
        for (int i = 1; i <= n; i++) fa[i] = i;
    }

    int find(int x) {
        if (fa[x] == x) return x;
        return fa[x] = find(fa[x]); // 路径压缩
    }

    void merge(int x, int y) {
        x = find(x); y = find(y);
        if (x == y) return;
        if (siz[x] > siz[y]) {
            fa[y] = x;
            siz[x] += siz[y];
        } else {
            fa[x] = y;
            siz[y] += siz[x];
        }
    }

    bool same(int x, int y) {
        return find(x) == find(y);
    }
};

```

种类并查集 (判断朋友/敌人)

```

struct DSU_Enemy {
    vector<int> fa, siz;
    DSU_Enemy(int n) : fa(2 * n + 1), siz(2 * n + 1, 1) {
        for (int i = 1; i <= 2 * n; i++) fa[i] = i;
    }

    int find(int x) {
        return fa[x] == x ? x : fa[x] = find(fa[x]);
    }

    void merge(int x, int y) {
        x = find(x); y = find(y);
        if (x == y) return;
        if (siz[x] > siz[y]) fa[y] = x, siz[x] += siz[y];
        else fa[x] = y, siz[y] += siz[x];
    }
};

```

```
// 使用:
// merge(x, y) 表示 x 和 y 是朋友
// merge(x + n, y) 和 merge(y + n, x) 表示 x 和 y 是敌人
```

2. 树状数组 (Fenwick Tree)

```
struct Fenwick {
    int n;
    vector<int> bit;

    Fenwick(int n_) : n(n_), bit(n_ + 1, 0) {}

    // 单点修改: 在 idx 位置增加 delta
    void add(int idx, int delta) {
        while (idx <= n) {
            bit[idx] += delta;
            idx += idx & -idx;
        }
    }

    // 前缀和查询: [1, idx] 的和
    int sum(int idx) {
        int s = 0;
        while (idx > 0) {
            s += bit[idx];
            idx -= idx & -idx;
        }
        return s;
    }

    // 区间查询: [l, r] 的和
    int range_sum(int l, int r) {
        return sum(r) - sum(l - 1);
    }
};
```

3. 线段树 (Segment Tree)

基础版 (区间求和, 单点修改)

```
struct SegTree {
    int n;
    vector<int> tree;

    SegTree(int n_) : n(n_), tree(4 * n_, 0) {}

    void update(int idx, int val) {
        update(1, 0, n - 1, idx, val);
    }
};
```

```

    }

    void update(int node, int l, int r, int idx, int val) {
        if (l == r) { tree[node] = val; return; }
        int mid = (l + r) / 2;
        if (idx <= mid) update(node * 2, l, mid, idx, val);
        else update(node * 2 + 1, mid + 1, r, idx, val);
        tree[node] = tree[node * 2] + tree[node * 2 + 1];
    }

    int query(int ql, int qr) {
        return query(1, 0, n - 1, ql, qr);
    }

    int query(int node, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr) return tree[node];
        int mid = (l + r) / 2, res = 0;
        if (ql <= mid) res += query(node * 2, l, mid, ql, qr);
        if (qr > mid) res += query(node * 2 + 1, mid + 1, r, ql, qr);
        return res;
    }
};

```

懒惰传播版（区间修改 + 区间求和）

```

struct SegTreeLazy {
    int n;
    vector<long long> tree, lazy; // long long 防溢出

    SegTreeLazy(int n_) : n(n_), tree(4 * n_, 0LL), lazy(4 * n_, 0LL) {}
    SegTreeLazy(const vector<long long>& a) : n(a.size()), tree(4 * n, 0LL),
    lazy(4 * n, 0LL) {
        build(1, 0, n - 1, a);
    }

    void build(int node, int l, int r, const vector<long long>& a) {
        if (l == r) { tree[node] = a[l]; return; }
        int mid = (l + r) / 2;
        build(node * 2, l, mid, a);
        build(node * 2 + 1, mid + 1, r, a);
        tree[node] = tree[node * 2] + tree[node * 2 + 1];
    }

    void push(int node, int l, int r) {
        if (lazy[node]) {
            tree[node] += lazy[node] * (r - l + 1);
            if (l != r) {
                lazy[node * 2] += lazy[node];
                lazy[node * 2 + 1] += lazy[node];
            }
            lazy[node] = 0;
        }
    }
};

```

```

void update(int ql, int qr, long long val) {
    update(1, 0, n - 1, ql, qr, val);
}

void update(int node, int l, int r, int ql, int qr, long long val) {
    push(node, l, r);
    if (ql > r || qr < l) return;
    if (ql <= l && r <= qr) { lazy[node] += val; push(node, l, r); return; }
    int mid = (l + r) / 2;
    update(node * 2, l, mid, ql, qr, val);
    update(node * 2 + 1, mid + 1, r, ql, qr, val);
    tree[node] = tree[node * 2] + tree[node * 2 + 1];
}

long long query(int ql, int qr) { return query(1, 0, n - 1, ql, qr); }
long long query(int node, int l, int r, int ql, int qr) {
    push(node, l, r);
    if (ql > r || qr < l) return 0;
    if (ql <= l && r <= qr) return tree[node];
    int mid = (l + r) / 2;
    return query(node * 2, l, mid, ql, qr) + query(node * 2 + 1, mid + 1, r,
ql, qr);
}
};

```

4. 平衡树

Treap (树堆)

```

// Treap: 结合堆和二叉搜索树的性质
// 期望时间复杂度: O(log n)
struct Treap {
    struct Node {
        int val;           // 节点值
        int priority;       // 随机优先级 (堆性质)
        int cnt;           // 该值出现的次数
        int size;          // 子树大小
        Node* lch;         // 左儿子
        Node* rch;         // 右儿子

        Node(int v) : val(v), priority(rand()), cnt(1), size(1), lch(nullptr),
rch(nullptr) {}
    };

    Node* root = nullptr;

    // 更新子树大小
    void pushup(Node* node) {
        if (node) {
            int left_size = node->lch ? node->lch->size : 0;
            int right_size = node->rch ? node->rch->size : 0;
            node->size = left_size + right_size + node->cnt;
        }
    }
}

```

```

}

// 右旋 (zig)
void rotate_right(Node*& node) {
    Node* left = node->lch;
    node->lch = left->rch;
    left->rch = node;
    node = left;
    pushup(node->rch);
    pushup(node);
}

// 左旋 (zag)
void rotate_left(Node*& node) {
    Node* right = node->rch;
    node->rch = right->lch;
    right->lch = node;
    node = right;
    pushup(node->lch);
    pushup(node);
}

// 插入值v
void insert(Node*& node, int v) {
    if (!node) {
        node = new Node(v);
        return;
    }

    if (v == node->val) {
        node->cnt++; // 值已存在, 计数+1
    } else if (v < node->val) {
        insert(node->lch, v);
        if (node->lch->priority > node->priority) {
            rotate_right(node); // 维护堆性质
        }
    } else {
        insert(node->rch, v);
        if (node->rch->priority > node->priority) {
            rotate_left(node); // 维护堆性质
        }
    }
    pushup(node);
}

// 删除值v
void remove(Node*& node, int v) {
    if (!node) return;

    if (v == node->val) {
        if (node->cnt > 1) {
            node->cnt--; // 计数-1
        } else {
            // 删除节点

```

```

        if (!node->lch && !node->rch) {
            delete node;
            node = nullptr;
        } else if (!node->lch) {
            Node* temp = node;
            node = node->rch;
            delete temp;
        } else if (!node->rch) {
            Node* temp = node;
            node = node->lch;
            delete temp;
        } else {
            // 两个儿子，旋转到叶子
            if (node->lch->priority > node->rch->priority) {
                rotate_right(node);
                remove(node->rch, v);
            } else {
                rotate_left(node);
                remove(node->lch, v);
            }
        }
    }
} else if (v < node->val) {
    remove(node->lch, v);
} else {
    remove(node->rch, v);
}
pushup(node);
}

// 查询v的排名（第几小）
int get_rank(Node* node, int v) {
    if (!node) return 1;

    int left_size = node->lch ? node->lch->size : 0;

    if (v == node->val) {
        return left_size + 1;
    } else if (v < node->val) {
        return get_rank(node->lch, v);
    } else {
        return left_size + node->cnt + get_rank(node->rch, v);
    }
}

// 查询第k小的值
int get_kth(Node* node, int k) {
    if (!node) return 0;

    int left_size = node->lch ? node->lch->size : 0;

    if (k <= left_size) {
        return get_kth(node->lch, k);
    } else if (k <= left_size + node->cnt) {

```

```

        return node->val;
    } else {
        return get_kth(node->rch, k - left_size - node->cnt);
    }
}

// 查询前驱（小于v的最大值）
int get_prev(Node* node, int v) {
    if (!node) return -INF;

    if (node->val >= v) {
        return get_prev(node->lch, v);
    } else {
        int right_prev = get_prev(node->rch, v);
        return max(node->val, right_prev);
    }
}

// 查询后继（大于v的最小值）
int get_next(Node* node, int v) {
    if (!node) return INF;

    if (node->val <= v) {
        return get_next(node->rch, v);
    } else {
        int left_next = get_next(node->lch, v);
        return min(node->val, left_next);
    }
}

// 公共接口
void insert(int v) { insert(root, v); }
void remove(int v) { remove(root, v); }
int get_rank(int v) { return get_rank(root, v); }
int get_kth(int k) { return get_kth(root, k); }
int get_prev(int v) { return get_prev(root, v); }
int get_next(int v) { return get_next(root, v); }
};

```

Splay树

```

// Splay树：通过旋转将访问的节点移到根
// 摊还时间复杂度：O(log n)
struct Splay {
    struct Node {
        int val;           // 节点值
        int cnt;           // 值出现次数
        int size;          // 子树大小
        Node* lch;         // 左儿子
        Node* rch;         // 右儿子
        Node* fa;          // 父亲
    };
};

```

```

        Node(int v) : val(v), cnt(1), size(1), lch(nullptr), rch(nullptr),
fa(nullptr) {}
    };

    Node* root = nullptr;

    // 判断node是父亲的左儿子还是右儿子
    bool is_left(Node* node) {
        return node->fa->lch == node;
    }

    // 更新子树大小
    void pushup(Node* node) {
        if (node) {
            int left_size = node->lch ? node->lch->size : 0;
            int right_size = node->rch ? node->rch->size : 0;
            node->size = left_size + right_size + node->cnt;
        }
    }

    // 旋转（将node向上旋转一层）
    void rotate(Node* node) {
        Node* parent = node->fa;
        Node* grand = parent->fa;

        if (is_left(node)) {
            // node是左儿子，右旋
            parent->lch = node->rch;
            if (node->rch) node->rch->fa = parent;
            node->rch = parent;
        } else {
            // node是右儿子，左旋
            parent->rch = node->lch;
            if (node->lch) node->lch->fa = parent;
            node->lch = parent;
        }

        parent->fa = node;
        node->fa = grand;

        if (grand) {
            if (grand->lch == parent) grand->lch = node;
            else grand->rch = node;
        }

        pushup(parent);
        pushup(node);
    }

    // Splay操作：将node旋转到target的儿子
    void splay(Node* node, Node* target = nullptr) {
        while (node->fa != target) {
            Node* parent = node->fa;

```



```

        Node* grand = parent->fa;

        if (grand != target) {
            if (is_left(node) == is_left(parent)) {
                // 同向：先转parent
                rotate(parent);
            } else {
                // 反向：先转node
                rotate(node);
            }
        }
        rotate(node);
    }

    if (!target) root = node;
}

// 查找值为v的节点，不存在则找插入位置
Node* find(int v) {
    Node* cur = root;
    Node* parent = nullptr;

    while (cur && cur->val != v) {
        parent = cur;
        if (v < cur->val) cur = cur->lch;
        else cur = cur->rch;
    }

    if (cur) {
        splay(cur); // 找到了，旋转到根
        return cur;
    }
    return nullptr;
}

// 插入值v
void insert(int v) {
    if (!root) {
        root = new Node(v);
        return;
    }

    Node* cur = root;
    Node* parent = nullptr;

    while (cur && cur->val != v) {
        parent = cur;
        if (v < cur->val) cur = cur->lch;
        else cur = cur->rch;
    }

    if (cur) {
        cur->cnt++; // 值已存在
    } else {

```

```

        cur = new Node(v);
        cur->fa = parent;
        if (v < parent->val) parent->lch = cur;
        else parent->rch = cur;
    }

    splay(cur);
}

// 删除值v
void remove(int v) {
    Node* node = find(v);
    if (!node) return;

    if (node->cnt > 1) {
        node->cnt--;
        pushup(node);
        return;
    }

    if (!node->lch && !node->rch) {
        // 无儿子
        root = nullptr;
        delete node;
    } else if (!node->lch) {
        // 只有右儿子
        root = node->rch;
        root->fa = nullptr;
        delete node;
    } else if (!node->rch) {
        // 只有左儿子
        root = node->lch;
        root->fa = nullptr;
        delete node;
    } else {
        // 两个儿子：找前驱替代
        Node* prev = node->lch;
        while (prev->rch) prev = prev->rch;

        splay(prev, node); // 将前驱旋转到node的左儿子

        prev->rch = node->rch;
        node->rch->fa = prev;
        root = prev;
        root->fa = nullptr;
        pushup(root);

        delete node;
    }
}

// 查询v的排名
int get_rank(int v) {
    find(v);

```

```

        if (!root) return 1;
        return root->lch ? root->lch->size + 1 : 1;
    }

    // 查询第k小的值
    int get_kth(int k) {
        Node* cur = root;

        while (cur) {
            int left_size = cur->lch ? cur->lch->size : 0;

            if (k <= left_size) {
                cur = cur->lch;
            } else if (k <= left_size + cur->cnt) {
                splay(cur);
                return cur->val;
            } else {
                k -= left_size + cur->cnt;
                cur = cur->rch;
            }
        }

        return 0;
    }

    // 查询前驱
    int get_prev(int v) {
        insert(v);
        Node* cur = root->lch;
        while (cur && cur->rch) cur = cur->rch;
        remove(v);
        return cur ? cur->val : -INF;
    }

    // 查询后继
    int get_next(int v) {
        insert(v);
        Node* cur = root->rch;
        while (cur && cur->lch) cur = cur->lch;
        remove(v);
        return cur ? cur->val : INF;
    }
};

```

5. 单调栈 / 单调队列

单调栈 (求每个元素左右第一个比它大/小的元素)

```

// 单调递增栈: 找到左边第一个比它小的元素
vector<int> left_smaller(const vector<int>& a) {
    int n = a.size();

```

```

vector<int> res(n, -1);
stack<int> stk;
for (int i = 0; i < n; i++) {
    while (!stk.empty() && a[stk.top()] >= a[i]) stk.pop();
    if (!stk.empty()) res[i] = stk.top();
    stk.push(i);
}
return res;
}

```

单调队列（滑动窗口最值）

```

// 求滑动窗口最大值
vector<int> sliding_window_max(vector<int>& a, int k) {
    deque<int> dq;
    vector<int> res;
    for (int i = 0; i < a.size(); i++) {
        if (!dq.empty() && dq.front() < i - k + 1) dq.pop_front();
        while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back();
        dq.push_back(i);
        if (i >= k - 1) res.push_back(a[dq.front()]);
    }
    return res;
}

```

第三部分：图论算法

图论

1. 图的存储

```

// 邻接表（稀疏图）
vector<int> adj[N]; // 无权图
vector<pair<int, int>> adj[N]; // 有权图 (to, weight)

// 链式前向星（最省内存，适合大图）
struct Edge { int to, next, w; };
Edge edges[N * 2];
int head[N], cnt;
void addEdge(int u, int v, int w = 1) {
    edges[++cnt] = {v, head[u], w};
    head[u] = cnt;
}

```

2. BFS (广度优先搜索)

```
void bfs(int s) {
    queue<int> qe;
    vector<bool> vis(n + 1, false);
    vector<int> dist(n + 1, INF);

    qe.push(s);
    vis[s] = true;
    dist[s] = 0;

    while (!qe.empty()) {
        int u = qe.front(); qe.pop();
        for (auto v : adj[u]) {
            if (!vis[v]) {
                vis[v] = true;
                dist[v] = dist[u] + 1;
                qe.push(v);
            }
        }
    }
}
```

棋盘 BFS (如骑士走法)

```
const int dx[8] = {1, 2, 2, 1, -1, -2, -2, -1};
const int dy[8] = {-2, -1, 1, 2, 2, 1, -1, -2};

void bfs_knight(int n, int m, int sx, int sy) {
    vector<vector<int>> dist(n + 1, vector<int>(m + 1, -1));
    queue<pair<int, int>> qe;

    qe.push({sx, sy});
    dist[sx][sy] = 0;

    while (!qe.empty()) {
        auto [x, y] = qe.front(); qe.pop();
        for (int i = 0; i < 8; i++) {
            int nx = x + dx[i], ny = y + dy[i];
            if (nx >= 1 && nx <= n && ny >= 1 && ny <= m && dist[nx][ny] == -1) {
                dist[nx][ny] = dist[x][y] + 1;
                qe.push({nx, ny});
            }
        }
    }
}
```

3. DFS (深度优先搜索)

```

vector<int> adj[N];
vector<bool> vis(N);

void dfs(int u) {
    vis[u] = true;
    for (int v : adj[u]) {
        if (!vis[v]) {
            dfs(v);
        }
    }
}

```

4. Dijkstra (最短路)

```

struct Edge { int to; ll w; };
vector<Edge> adj[N];
vector<ll> dist(N, INF);
vector<bool> vis(N, false);

void dijkstra(int s) {
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>>
pq;
    dist[s] = 0;
    pq.push({0, s});

    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (vis[u]) continue;
        vis[u] = true;

        for (auto& [v, w] : adj[u]) {
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
}

```

5. Floyd-Warshall (多源最短路)

```

void floyd() {
    for (int k = 1; k <= n; k++) {
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                if (dist[i][k] != INF && dist[k][j] != INF) {
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
                }
            }
        }
    }
}

```

```

    }
  }
}

```

6. 最小生成树 (MST)

Kruskal 算法

```

// 边结构体：存储边的两个端点和权值
struct Edge {
    int u, v, w; // u: 起点, v: 终点, w: 权值
};

// 比较函数：按边权从小到大排序
bool cmp(const Edge& a, Edge& b) {
    return a.w < b.w;
}

// Kruskal算法求最小生成树
// 时间复杂度: O(m log m), m为边数
ll kruskal(int n, vector<Edge>& edges) {
    // 步骤1: 按边权排序 (贪心策略)
    sort(edges.begin(), edges.end(), cmp);

    // 步骤2: 初始化并查集
    DSU dsu(n);

    ll mst_cost = 0; // MST总权值
    int cnt = 0;    // 已选边数

    // 步骤3: 依次考察每条边
    for (auto& e : edges) {
        // 如果这条边的两个端点不在同一连通块
        if (dsu.merge(e.u, e.v)) {
            mst_cost += e.w; // 加入MST
            cnt++;          // 边数+1
        }
        // MST有n-1条边, 选够就停止
        if (cnt == n - 1) break;
    }

    // 返回MST总权值, 若图不连通返回-1
    return (cnt == n - 1) ? mst_cost : -1;
}

```

Prim 算法

```

// Prim算法求最小生成树（适合稠密图）
// 时间复杂度:  $O(m \log n)$  或  $O(n^2)$ （邻接矩阵）
ll prim(int n, int s, vector<pair<int,int>> adj[]) {
    // dist[i]: 点i到当前MST的最短距离
    vector<ll> dist(n + 1, LINF);
    vector<bool> vis(n + 1, false); // 是否已加入MST

    // 优先队列: (距离, 点), 小根堆
    priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<pair<ll,int>>> pq;

    dist[s] = 0; // 起点到MST距离为0
    pq.push({0, s}); // 起点入队

    ll mst_cost = 0; // MST总权值
    int cnt = 0; // 已选点数

    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop(); // 取距离最小的点

        if (vis[u]) continue; // 已访问则跳过
        vis[u] = true; // 标记为已访问
        mst_cost += d; // 累加权值
        cnt++; // 点数+1

        // 遍历u的所有邻居
        for (auto& [v, w] : adj[u]) {
            // 如果v未访问, 且通过u到MST的距离更短
            if (!vis[v] && dist[v] > w) {
                dist[v] = w; // 更新距离
                pq.push({dist[v], v}); // 入队
            }
        }
    }

    // 返回MST总权值, 若图不连通返回-1
    return (cnt == n) ? mst_cost : -1;
}

```

7. 生成树扩展

次小生成树

```

// 次小生成树: 权值第二小的生成树
// 思路: 先求MST, 再枚举删除MST中的每条边, 求剩余图的最小生成树
struct Edge {
    int u, v, w;
    bool in_mst; // 是否在MST中
};

int second_mst(int n, vector<Edge>& edges) {

```



```

// 步骤1: 求MST
sort(edges.begin(), edges.end(), [](Edge& a, Edge& b) {
    return a.w < b.w;
});

DSU dsu(n);
int mst_cost = 0;
vector<int> mst_edges; // MST中边的下标

for (int i = 0; i < edges.size(); i++) {
    if (dsu.merge(edges[i].u, edges[i].v)) {
        mst_cost += edges[i].w;
        edges[i].in_mst = true;
        mst_edges.push_back(i);
    }
}

// 步骤2: 枚举删除MST中的每条边
int second_cost = LINF;

for (int del_idx : mst_edges) {
    // 重新建图, 不包含删除的边
    DSU dsu2(n);
    int cost = 0;
    int cnt = 0;

    for (int i = 0; i < edges.size(); i++) {
        if (i == del_idx) continue; // 跳过删除的边

        if (dsu2.merge(edges[i].u, edges[i].v)) {
            cost += edges[i].w;
            cnt++;
        }
    }

    if (cnt == n - 1) { // 找到生成树
        second_cost = min(second_cost, cost);
    }
}

return second_cost;
}

```

最小生成树计数

```

// 统计最小生成树的个数
// 思路: 按权值分组, 对每组边使用矩阵树定理
// 这里给出简化版: 假设所有边权值不同 (唯一MST)
const int MOD = 1e9 + 7;

int count_mst(int n, vector<Edge>& edges) {

```

```

// 如果所有边权值不同，MST唯一
// 返回1即可
// 如果有相同权值的边，需要用矩阵树定理

sort(edges.begin(), edges.end(), [](Edge& a, Edge& b) {
    return a.w < b.w;
});

DSU dsu(n);
int cnt = 0;
ll total_w = 0;

for (auto& e : edges) {
    if (dsu.merge(e.u, e.v)) {
        cnt++;
        total_w += e.w;
    }
}

return (cnt == n - 1) ? 1 : 0; // 简化版
}

```

7. 拓扑排序

```

vector<int> topological_sort(int n, vector<int> adj[]) {
    vector<int> in_degree(n + 1, 0);
    for (int u = 1; u <= n; u++) {
        for (int v : adj[u]) {
            in_degree[v]++;
        }
    }

    queue<int> qe;
    for (int i = 1; i <= n; i++) {
        if (in_degree[i] == 0) qe.push(i);
    }

    vector<int> topo_order;
    while (!qe.empty()) {
        int u = qe.front(); qe.pop();
        topo_order.push_back(u);

        for (int v : adj[u]) {
            in_degree[v]--;
            if (in_degree[v] == 0) qe.push(v);
        }
    }

    if (topo_order.size() != n) {
        return {}; // 存在环
    }
}

```

```
    return topo_order;
}
```

8. LCA (最近公共祖先)

```
const int MAXLOG = 20;
vector<int> adj[N];
int parent[N][MAXLOG], depth[N];

void dfs_lca(int u, int p, int d) {
    parent[u][0] = p;
    depth[u] = d;
    for (int i = 1; i < MAXLOG; i++) {
        parent[u][i] = parent[parent[u][i - 1]][i - 1];
    }
    for (int v : adj[u]) {
        if (v != p) dfs_lca(v, u, d + 1);
    }
}

int lca(int u, int v) {
    if (depth[u] < depth[v]) swap(u, v);
    int diff = depth[u] - depth[v];
    for (int i = 0; i < MAXLOG; i++) {
        if (diff & (1 << i)) u = parent[u][i];
    }
    if (u == v) return u;
    for (int i = MAXLOG - 1; i >= 0; i--) {
        if (parent[u][i] != parent[v][i]) {
            u = parent[u][i];
            v = parent[v][i];
        }
    }
    return parent[u][0];
}
```

第四部分：搜索与DP

搜索算法

1. 二分查找

手写二分

```
// 查找第一个 >= x 的位置 (lower_bound)
int lower_bound_custom(vector<int>& v, int x) {
    int l = 0, r = v.size();
    while (l < r) {
        int mid = (l + r) / 2;
        if (v[mid] >= x) r = mid;
        else l = mid + 1;
    }
    return l;
}

// 查找第一个 > x 的位置 (upper_bound)
int upper_bound_custom(vector<int>& v, int x) {
    int l = 0, r = v.size();
    while (l < r) {
        int mid = (l + r) / 2;
        if (v[mid] > x) r = mid;
        else l = mid + 1;
    }
    return l;
}
```

2. 浮点数二分 (二分答案)

```
double binary_search_double(double l, double r) {
    for (int i = 0; i < 100; i++) { // 迭代100次, 精度足够
        double mid = (l + r) / 2;
        if (check(mid)) r = mid;
        else l = mid;
    }
    return (l + r) / 2;
}
```

3. 三分搜索 (求单峰函数极值)

```
double ternary_search(double l, double r) {
    for (int i = 0; i < 100; i++) {
        double m1 = l + (r - l) / 3;
        double m2 = r - (r - l) / 3;
        if (f(m1) < f(m2)) l = m1;
        else r = m2;
    }
    return (l + r) / 2;
}
```

动态规划

1. 0/1 背包

```
int knapsack_01(int n, int V, vector<int>& w, vector<int>& v) {  
    vector<int> dp(V + 1, 0);  
    for (int i = 1; i <= n; i++) {  
        for (int j = V; j >= w[i]; j--) {  
            dp[j] = max(dp[j], dp[j - w[i]] + v[i]);  
        }  
    }  
    return dp[V];  
}
```

2. 完全背包

```
int knapsack_complete(int n, int V, vector<int>& w, vector<int>& v) {  
    vector<int> dp(V + 1, 0);  
    for (int i = 1; i <= n; i++) {  
        for (int j = w[i]; j <= V; j++) {  
            dp[j] = max(dp[j], dp[j - w[i]] + v[i]);  
        }  
    }  
    return dp[V];  
}
```

3. 最长上升子序列 (LIS)

```
int lis(vector<int>& a) {  
    vector<int> dp;  
    for (int x : a) {  
        auto it = lower_bound(dp.begin(), dp.end(), x);  
        if (it == dp.end()) dp.push_back(x);  
        else *it = x;  
    }  
    return dp.size();  
}
```

4. 最大子段和

```
int max_subarray_sum(vector<int>& nums) {  
    int max_sum = nums[0], cur_sum = nums[0];  
    for (int i = 1; i < nums.size(); i++) {  
        cur_sum = max(nums[i], cur_sum + nums[i]);  
        max_sum = max(max_sum, cur_sum);  
    }  
}
```

```
    }  
    return max_sum;  
}
```

第五部分：数学与字符串

数学与数论

1. 快速幂

```
ll fast_pow(ll a, ll b, ll m) {  
    ll res = 1 % m;  
    while (b) {  
        if (b & 1) res = res * a % m;  
        a = a * a % m;  
        b >>= 1;  
    }  
    return res;  
}
```

2. 埃氏筛（素数筛）

```
vector<bool> sieve(int n) {  
    vector<bool> is_prime(n + 1, true);  
    is_prime[0] = is_prime[1] = false;  
    for (int i = 2; i * i <= n; i++) {  
        if (is_prime[i]) {  
            for (int j = i * i; j <= n; j += i) {  
                is_prime[j] = false;  
            }  
        }  
    }  
    return is_prime;  
}
```

3. 欧几里得算法（GCD）

```
int gcd(int a, int b) {  
    return b == 0 ? a : gcd(b, a % b);  
}  
  
int lcm(int a, int b) {
```

```
    return a / gcd(a, b) * b;  
}
```

4. 扩展欧几里得算法

```
int exgcd(int a, int b, int& x, int& y) {  
    if (b == 0) { x = 1; y = 0; return a; }  
    int g = exgcd(b, a % b, y, x);  
    y -= a / b * x;  
    return g;  
}
```

字符串

1. KMP 算法

```
vector<int> get_next(string& pattern) {  
    int n = pattern.size();  
    vector<int> nxt(n + 1, 0);  
    for (int i = 1; i < n; i++) {  
        int j = nxt[i];  
        while (j > 0 && pattern[i] != pattern[j]) j = nxt[j];  
        if (pattern[i] == pattern[j]) j++;  
        nxt[i + 1] = j;  
    }  
    return nxt;  
}  
  
int kmp(string& text, string& pattern) {  
    vector<int> nxt = get_next(pattern);  
    int cnt = 0, j = 0;  
    for (int i = 0; i < text.size(); i++) {  
        while (j > 0 && text[i] != pattern[j]) j = nxt[j];  
        if (text[i] == pattern[j]) j++;  
        if (j == pattern.size()) {  
            cnt++;  
            j = nxt[j];  
        }  
    }  
    return cnt;  
}
```

2. 字符串哈希

```
struct StringHash {
    string s;
    vector<ull> h;
    ull base = 131;

    StringHash(string& str) : s(str), h(str.size() + 1, 0) {
        for (int i = 0; i < s.size(); i++) {
            h[i + 1] = h[i] * base + s[i];
        }
    }

    ull get_hash(int l, int r) {
        return h[r + 1] - h[l] * pow_base(r - l + 1);
    }

    ull pow_base(int k) {
        ull res = 1;
        while (k--) res *= base;
        return res;
    }
};
```

第六部分：技巧与实战

常用函数与技巧

1. 前缀和

```
vector<int> prefix_sum(vector<int>& a) {
    int n = a.size();
    vector<int> prefix(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        prefix[i] = prefix[i - 1] + a[i - 1];
    }
    return prefix;
}

// 查询区间和 [l, r]
int range_sum(vector<int>& prefix, int l, int r) {
    return prefix[r + 1] - prefix[l];
}
```

2. 差分


```
// 一维差分
vector<int> prefix(n + 2, 0);
for (auto& [l, r, val] : queries) {
    prefix[l] += val;
    prefix[r + 1] -= val;
}
for (int i = 1; i <= n; i++) {
    diff[i] = diff[i - 1] + prefix[i];
}
```

3. 离散化

```
vector<int> discretize(vector<int>& a) {
    vector<int> sorted = a;
    sort(sorted.begin(), sorted.end());
    sorted.erase(unique(sorted.begin(), sorted.end()), sorted.end());

    vector<int> res(a.size());
    for (int i = 0; i < a.size(); i++) {
        res[i] = lower_bound(sorted.begin(), sorted.end(), a[i]) - sorted.begin()
+ 1;
    }
    return res;
}
```

4. 高精度运算

高精度加法

```
string add(string a, string b) {
    string res;
    int carry = 0;
    int i = a.size() - 1, j = b.size() - 1;
    while (i >= 0 || j >= 0 || carry) {
        int sum = carry;
        if (i >= 0) sum += a[i--] - '0';
        if (j >= 0) sum += b[j--] - '0';
        res += (sum % 10) + '0';
        carry = sum / 10;
    }
    reverse(res.begin(), res.end());
    return res;
}
```

5. 位运算技巧

```
// 判断奇偶
if (x & 1) { /* 奇数 */ }

// lowbit（树状数组常用）
int lowbit(int x) { return x & -x; }

// 统计1的个数
int count_ones(int x) { return __builtin_popcount(x); }

// 判断是否为2的幂
bool is_power_of_two(int x) { return x > 0 && (x & (x - 1)) == 0; }
```

洛谷模板题验证列表

以下是常用模板对应的洛谷题目，用于验证模板正确性：

算法	洛谷题目	难度
BFS（棋盘）	P1443 骑士遍历	普及/提高-
Dijkstra	P1135 奇怪的电梯	普及/提高-
DSU	P3367 【模板】并查集	普及/提高-
拓扑排序	P1114 "非常男女"	普及/提高-
二分查找	P1102 A-B 数对	普及/提高-
前缀和	P1115 最大子段和	普及/提高-
01背包	P1048 采药	普及/提高-
素数筛	P1217 [USACO1.5] 回文质数	普及/提高-
KMP	P3375 【模板】KMP	普及+/提高
LCA	P3379 【模板】最近公共祖先	普及+/提高
线段树	P3372 【模板】线段树	普及+/提高

比赛常用技巧

1. 输入输出优化

```
// 关同步（已经在宏定义中）
ios::sync_with_stdio(false);
cin.tie(nullptr); cout.tie(nullptr);

// 使用 getchar 快读
inline int read() {
    int x = 0, f = 1; char ch = getchar();
```

```
while (ch < '0' || ch > '9') { if (ch == '-') f = -1; ch = getchar(); }
while (ch >= '0' && ch <= '9') { x = x * 10 + ch - '0'; ch = getchar(); }
return x * f;
}
```

2. 调试技巧

```
#ifdef LOCAL
#define debug(x) cerr << #x << " = " << x << endl
#else
#define debug(x)
#endif

// 使用
debug(dist[u]);
```

3. 常用常数

```
const int MOD = 1e9 + 7;
const int MOD = 998244353;
const int INF = 0x3f3f3f3f; // 约 1e9
const ll LINF = 0x3f3f3f3f3f3f3f3f;
const double EPS = 1e-9;
```
